Pilot report: SemTrace-Dep, context conditions C0–C3

Local model: **qwen2.5-coder:7b** (Ollama 0.40.16, temperature 0) on a 16 GB Mac. Runs: 10–11 Nov 2026.

1. Question

Can a small code model compute `f(x)` for a target function whose answer depends on a short call chain, when the needed functions are:

- **C0**: given alone (upper bound),
- **C1**: scattered among distractor functions at relative position p = 0, 0.5 or 1,
- **C2**: placed as one block at the end of the same distractor context,
- **C3**: reduced to the static call-graph closure from the target (Python `ast`)?

RQ1: is C1@p=0.5 worse than C1@p=0 and p=1 (lost in the middle)? RQ2: do C2 or C3 recover accuracy toward C0?

2. Runs

Result files are `results/qwen2.5-coder_7b_<ID>.jsonl`.

Run	pilot.py	Difficulty	Prompt	Context	n	ID
v1	v1	depth 2, k 3, m 3, with *	direct answer	~4k	20	20261110_224433
calib (a)	v2	depth 2, k 3, m 2	direct	C0, C3	10	20261110_232922
calib (b)	v2	depth 1, k 2, m 1	direct	C0, C3	10	20261110_232950
calib (b) CoT	v2	depth 1, k 2, m 1	`--cot`	C0, C3	10	20261110_233022
calib (a) CoT	v2	depth 2, k 3, m 2	`--cot`	C0	10	20261110_233851
v2-CoT	v2	depth 2, k 3, m 2	`--cot`, max out 1024	~4k	20	20261111_012743
v3 (main)	v2 + `--max-out`	depth 2, k 3, m 2	`--cot`, max out 2048	`--tokens 2000`, seed 2	**50**	20261111_071717

The v2-CoT run was first interrupted after 12 items, when my previous session on the Mac ended. Its partial file 20261110_234502 is kept but not used in this report. The run was then restarted detached.

3. Calibration: direct answers do not work

Run	C0	C3
v1, direct (all conditions 0%)	0/20	0/20
(a) direct	0/10	0/10
(b) easiest, direct	0/10	0/10
(b) easiest, CoT	8/10	9/10
(a) default, CoT	7/10	—

With “answer with the literal only” the model outputs ~11 tokens and guesses (e.g. `[10, 10]` for `[-409, -709]`), even for one helper with one step. With step-by-step reasoning the default difficulty is solvable (C0 70%). All later runs use `--cot` and the default difficulty.

4. Main result (v3, n = 50)

Runtime 11 470 s (~3 h 11 min, ~229 s per item, 300 queries).

Condition	Correct	Accuracy	Wilson 95% CI
C0	34/50	68%	[54.2, 79.2]
C1 @ p=0	15/50	30%	[19.1, 43.8]
C1 @ p=0.5	16/50	32%	[20.8, 45.8]
C1 @ p=1	6/50	12%	[5.6, 23.8]
C2	13/50	26%	[15.9, 39.6]
C3	37/50	74%	[60.4, 84.1]

Paired comparisons, matched on item, exact two-sided McNemar (binomial) test. “A only” means right in A and wrong in B.

A vs B	A only	B only	p
C1@p=0 vs C1@p=0.5	11	12	1.0
C1@p=1 vs C1@p=0.5	5	15	0.041
C1@p=0 vs C1@p=1	13	4	0.049
C2 vs C1@p=0.5	8	11	0.65
C3 vs C1@p=0.5	24	3	4.9e-05
C0 vs C1@p=0.5	20	2	0.00012

Replication on ~4k context (v2-CoT, n = 20, max out 1024)

C0 65%, C1@p=0 25%, C1@p=0.5 5%, C1@p=1 5%, C2 15%, C3 75%. C3 vs C1@p=0.5: 14 / 0, p = 0.00012. Position comparisons were not significant (p=0 vs 0.5: 5 / 1, p = 0.22; p=1 vs 0.5: 1 / 1, p = 1.0). C1 was near the floor at this length.

5. Findings

1. **Distractors cost about 40 points.** C0 68% vs C1 12–32% (v3). The pattern is the same at ~4k context (C0 65% vs C1 5–25%).
2. **RQ1, no lost-in-the-middle U-shape.** p=0 and p=0.5 are equal (30% vs 32%, p = 1.0). The **end** position is the worst (12%), significantly below both the middle (p = 0.041) and the start (p = 0.049). At ~4k context all positions are at the floor, so they cannot be separated.
3. **RQ2, C3 recovers fully; C2 does not.** The static closure scores 74%, at the C0 level, and far above C1@p=0.5 (24 vs 3 discordant pairs, p < 0.001). Moving the needed block to the end (C2, 26%) does not help, which fits finding 2: for this model and prompt the end of the context is a weak spot, not a strong one.
4. **C3 is nearly exact on this synthetic code** (149/150 contexts in v3, 60/60 in v2-CoT). This is risk R4 from the plan: the code has no indirect calls, so a name-based call graph is enough.

6. Caveats and problems found

- **Unparsed answers.** v3: 32 of 300 (C2 10, C1@0 7, C1@1 6, C1@0.5 6, C3 2, C0 1), counted as wrong. 31 of the 32 hit the 2048-token output cap, and every answer that hit the cap is unparsed. Raising the cap from 1024 to 2048 did not reduce the rate: the model loops. One example repeats `fn_5986 is called with x = 3930 + 81 = 4011`, then `x = 4011 + 81 = 4092`, and so on. Most loops are in the long-context conditions, so they are part of the distractor effect, not noise.
- **Generator name collision (bug).** The single inexact C3 case (v3, item 39, p=0.5) happens because a distractor got the same random 4-hex name as the target, `fn_810b`. The C1 context then defines `fn_810b` twice. Under Python semantics the later definition wins: the context returns `918`, while the stored answer is `[512, 201]`. The ground truth is therefore wrong for that one context (the C1@p=0.5 and C3 rows of item 39). It changes at most one count in two cells. Fix: make `rand_name` reject names already used in the item and its distractors.
- **The token length is underestimated.** `approx_tokens = len/4`, so the measured prompts are about 3.7k tokens for `--tokens 2000` (C1/C2 max 3885) and about 350 tokens for C0/C3. num_ctx is 8192, so nothing was truncated. The p=1 result is therefore not a truncation artefact.
- **Small model, one seed per run, synthetic code.** A p-value of 0.04–0.05 with 50 items is suggestive, not conclusive. The end-position effect should be replicated with another seed and at least one other model before it is used as a claim.
- **The end position is next to the question.** With p=1 (and C2) the needed code is just before the repeated question. A plausible reading is that the model copies or loops on the most recent code rather than reading back. That is a hypothesis, not tested here.

7. Hardware and speed

Machine and software (measured)

Item	Value
Computer	MacBook Pro (Mac14,9)
Chip	Apple M2 Pro: 10-core CPU (6 performance + 4 efficiency), 16-core GPU (Metal 4)
Memory	16 GB unified (CPU and GPU share it), 200 GB/s peak bandwidth (Apple spec)
OS	macOS 27.0.1 (26A434)
Runtime	Ollama 0.40.16 (llama.cpp backend, Metal)
Model	qwen2.5-coder:7b, 7.6B parameters, Q4_K_M quantisation, 4.7 GB on disk
In memory	5.0 GB at num_ctx 8192, **100% GPU** (`ollama ps`)
Decoding	temperature 0, one query at a time (no batching)

Throughput (measured)

Two cold queries with fresh C1 contexts and the CoT prompt (Ollama reuses the KV cache for a repeated prompt, so repeated prompts give inflated numbers and are not used here):

Query	Prompt tokens	Prompt eval rate	Output tokens	Eval (generation) rate	Total
1	3666	274 tok/s	512	28.7 tok/s	31.3 s
2	3921	281 tok/s	512	30.1 tok/s	31.1 s

Main run (v3, 300 queries): 777 894 prompt tokens (mean 2593, median 3660 per query) and 257 316 output tokens (mean 858, median 728), in 11 470 s, so **38.2 s per query**. The model "prompt tokens / 277 + output tokens / 29.4" predicts 2808 s + 8752 s = 11 561 s, within 1% of the measured runtime. About **75% of the time is token generation** and 25% is reading the prompt.

Comparison with NVIDIA GPUs (estimated, not measured)

Generation with a 4-bit 7B model is limited by memory bandwidth: every new token reads the whole ~4.7 GB of weights. On the M2 Pro, 29.4 tok/s × 4.7 GB = 138 GB/s, which is 69% of the 200 GB/s peak. The table applies the same 69% efficiency to each card's peak bandwidth. Prompt reading is limited by compute, and NVIDIA cards with tensor cores are much faster there; the estimate assumes 10× the Mac's rate. The model needs about 5 GB, so it fits on every card listed.

GPU	VRAM	Peak bandwidth	Est. generation rate	Est. time for v3 (300 queries)
Apple M2 Pro (this Mac, measured)	16 GB shared	200 GB/s	**29 tok/s**	**191 min**
RTX 4060 Ti 16GB	16 GB	288 GB/s	~42 tok/s	~106 min
Tesla T4 (e.g. free Colab)	16 GB	320 GB/s	~47 tok/s	~96 min
RTX 3060 12GB	12 GB	360 GB/s	~53 tok/s	~86 min
RTX 3090	24 GB	936 GB/s	~138 tok/s	~36 min
RTX 4090	24 GB	1008 GB/s	~148 tok/s	~34 min
A100 80GB SXM	80 GB	2039 GB/s	~300 tok/s	~19 min

How to read this: a mid-range card (3060, 4060 Ti, T4) would be about 2× faster than this Mac, and a high-end consumer card (3090, 4090) about 5–6× faster. These are rough upper estimates for one query at a time. Real numbers depend on the driver and the llama.cpp build: the T4 and A100 usually come in below a linear scaling, and with a single stream the A100's large bandwidth is not fully used. A batched server (e.g. vLLM running many queries in parallel) would be much faster on any NVIDIA card, but would change nothing about accuracy. At temperature 0 the answers should be the same up to small numerical differences between backends. The hardware affects only runtime, not the accuracy results above.

8. Suggested next steps

1. Fix the name collision in `rand_name`, and use a real tokenizer count (or Ollama's `prompt_eval_count`) for `--tokens`.
2. Replicate v3 with `--seed 3` and a second model (e.g. `llama3.2:3b`, or a larger model if available) to check the end-position effect.
3. Add indirect calls (dispatch tables, `getattr`, callbacks) so C3 is no longer trivially exact (risk R4).
4. Optionally add a repetition penalty, or stop on a repeated line, to cut the looping answers. Keep it identical across conditions.

Files

- Scripts: `pilot_v1.py` (v1), `pilot_v2.py` (v2), `pilot.py` (v2 + `--max-out`).
- Logs: `run_4k.log`, `calib_a.log`, `calib_b.log`, `calib_b_cot.log`, `calib_a_cot.log`, `run_v2_cot_interrupted.log`, `run_v2_cot.log`, `run_v3_2k.log`.
- Each `results/*.jsonl` line holds the item, condition, position, expected answer, raw answer, parsed answer, correctness and token counts.